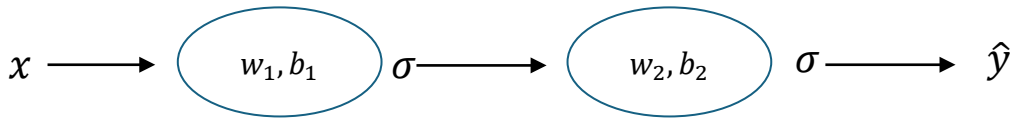
Introduction to Machine Learning

Session 2 — Lecture Notes

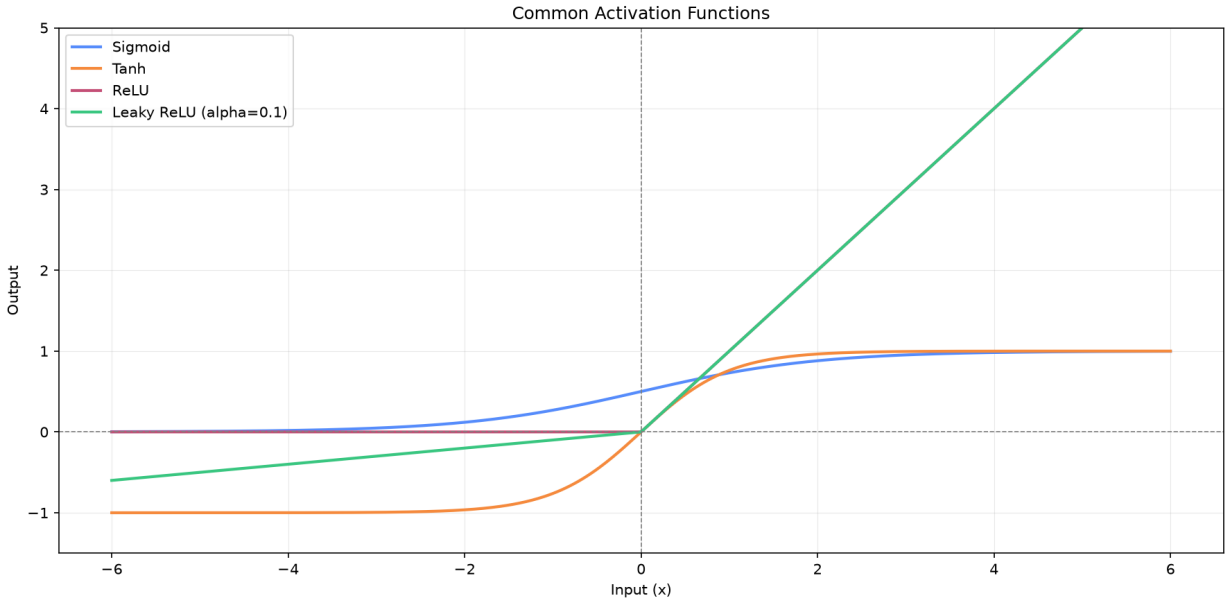
Introducing Non-Linearity

To change the perceptron into a non-linear function, we have to introduce non-linearity.



$\hat{y} = \sigma(\sigma(w_1 x + b_1)w_2 + b_2)$ *cannot be transformed into a linear model.*

Activation functions



Sigmoid

$$\sigma = \frac{1}{(1 + e^{-x})}$$

- Everything squashed into (0, 1) range.
- Slower learning, as it's not centred around 0.
- For large positives or negatives it's almost flat, so gradients vanish.

Tanh — Hyperbolic Tangent of z

$$tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

- Range (−1, 1), S-curve.
- Better at optimization but vanishing gradient still a problem.

ReLU — Rectified Linear Unit

$$\sigma(x) = max(0, x)$$

- Simple.
- Problem with negative inputs → neurons can get stuck.
- Insanely popular, as the gradient-vanishing problem is non-existent.

Leaky ReLU

$$\sigma(x) = \begin{cases} x, & if\ x \geq 0 \\ 0.1x, & if\ x < 0 \end{cases}$$

The slope argument is tunable.

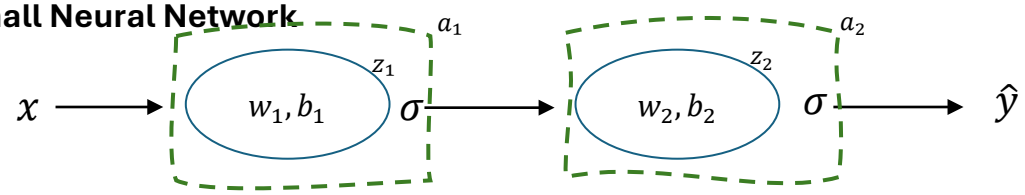
Softmax

$$\sigma(x_i) = \frac{e^{xi}}{\Sigma_{j=1}\, e^{xj}}$$

Not in this class, but that is for the output layer if we have multiple classes / output. The probabilities of all classes sum to 1.

Choosing an activation function is important! Hidden layers can use any, but the output needs to be paired with the loss function. The modern 'default' is ReLU on hidden layers, and sigmoid on output, as it pairs amazingly with the Binary Cross-Entropy function.

Small Neural Network



Forward pass

$$z_1 = w_1 x + b_1$$

$$a_1 = \sigma(z_1) = \frac{1}{1 + e^{-z_1}}$$

$$z_2 = w_2 a_1 + b_2$$

$$a_2 = \frac{1}{1 + e^{-z_2}} = \hat{y}$$

How to Train the Network?

Step 1 — Backpropagation.

Step 2 — Gradient descent

Backpropagation

We go backwards to assign the blame.

Loss Function — Binary Cross Entropy Loss

$$L = -[\, y \cdot log(\hat{y}) + (1 - y) \cdot log(1 - \hat{y})\,]$$

Note: log is ln, but this is typical usage in the AI community.

$\hat{y}$ is a probability of class 1, so a float number.

Sigmoid Activation Function (everywhere, for simplicity)

$$\sigma = \frac{1}{(1 + e^{-x})}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Now, Let's Derive parameters Influence on Loss

$\hat{y}$	$L = -[y \cdot log(\hat{y}) + (1-y) \cdot log(1-\hat{y})]$
z_2	$L = -[y \cdot log(\sigma(z_2)) + (1-y) \cdot log(1-\sigma(z_2))]$
$a_1, (w_2, b_2)$	$L = -[y \cdot log(\sigma(w_2 a_1 + b_2)) + (1-y) \cdot log(1-\sigma(w_2 a_1 + b_2))]$
z_1	$L = -[y \cdot log(\sigma(w_2 \sigma(z_1) + b_2)) + (1-y) \cdot log(1-\sigma(w_2 \sigma(z_1) + b_2))]$
(w_1, b_1)	$L = -[y \cdot log(\sigma(w_2 \sigma(w_1 x + b_1) + b_2)) + (1-y) \cdot log(1-\sigma(w_2 \sigma(w_1 x + b_1) + b_2))]$

To go backwards we use the CHAIN RULE:

$$\frac{d}{dx}f(g(x)) = \frac{df}{dg} \cdot \frac{dg}{dx}$$

In backpropagation, we use the chain rule to compute how much each parameter influenced the loss — the partial derivative of the loss with respect to each parameter. Together these partial derivatives form the gradient.

$\hat{y}$ $\quad \frac{\partial L}{\partial \hat{y}} = \frac{-y}{\hat{y}} + \frac{(1-y)}{(1-\hat{y})} = \frac{-y(1-\hat{y}) + (1-y)\hat{y}}{\hat{y}(1-\hat{y})} = \frac{(\hat{y}-y)}{\hat{y}(1-\hat{y})}$

z_2 $\quad \frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} = \frac{(\hat{y}-y)}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y}) = \hat{y} - y = \delta_2$

w_2 $\quad \frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_2} = \delta_2 \cdot a_1$

b_2 $\quad \frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z_2} \cdot \frac{\partial z_2}{\partial b_2} = \delta_2$

} *part of the gradient*

a_1 $\quad \frac{\partial L}{\partial a_1} = \frac{\partial L}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} = \delta_2 \cdot w_2$

z_1 $\quad \frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} = \delta_2 \cdot w_2 \cdot a_1(1-a_1) = \delta_1$

w_1 $\quad \frac{\partial L}{\partial w_1} = \partial L/\partial z_1 \cdot \partial z_1/\partial w_1 = \delta_1 \cdot x$

b_1 $\quad \partial L/\partial b_1 = \partial L/\partial z_1 \cdot \partial z_1/\partial b_1 = \delta_1 \cdot 1 = \delta_1$

} *part of the gradient*

Gradient

$$\nabla L = \left(\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b_2}\right)$$

$$\nabla L = (\delta_1 \cdot x, \delta_1, \delta_2 \cdot a_1, \delta_2)$$

Note! The calculations above are done for one data point only.

For the gradient calculations for entire dataset with n samples, the end formulas change into:

$$L = -\frac{1}{n}\sum_{i=1}^{n}[\, y \cdot log(\hat{y}) + (1-y) \cdot log(1-\hat{y})\,]$$

$$\nabla L = \left(\frac{1}{n}\sum_{i=1}^{n}\frac{\partial L_i}{\partial w_1}, \frac{1}{n}\sum_{i=1}^{n}\frac{\partial L_i}{\partial b_1}, \frac{1}{n}\sum_{i=1}^{n}\frac{\partial L_i}{\partial w_2}, \frac{1}{n}\sum_{i=1}^{n}\frac{\partial L_i}{\partial b_2}\right)$$

Gradient Descent

$$\nabla L = \left(\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b_2}\right)$$

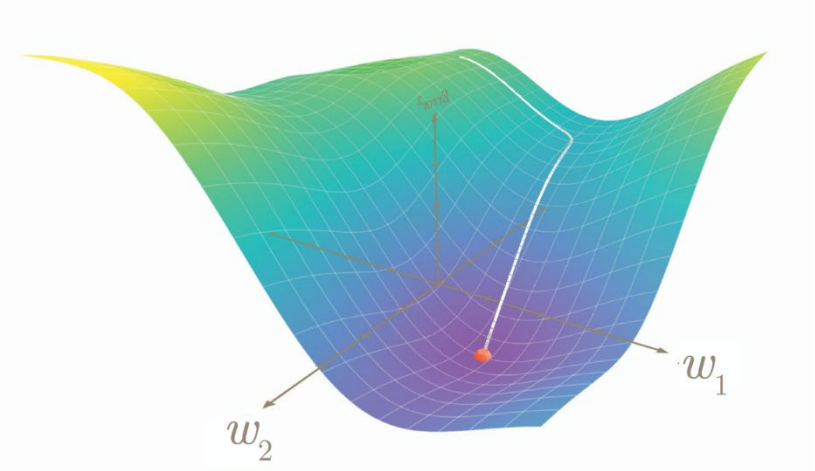
η = learning rate

$$w_{1\,new} = w_1 - \eta \cdot \frac{1}{n}\sum_{i=1}^{n} \frac{\partial L_i}{\partial w_1}$$

$$w_{2\,new} = w_2 - \eta \cdot \frac{1}{n}\sum_{i=1}^{n} \frac{\partial L_i}{\partial w_2}$$

$$b_{1\,new} = b_1 - \eta \cdot \frac{1}{n}\sum_{i=1}^{n} \frac{\partial L_i}{\partial b_1}$$

$$b_{2\,new} = b_2 - \eta \cdot \frac{1}{n}\sum_{i=1}^{n} \frac{\partial L_i}{\partial b_2}$$



Source: Welch labs, Illustrated Guide to AI

How to Choose η?

- Check values 0.01–1, 0.1 as first try.
- Loss plateaus immediately → η too small
- Loss oscillates widely → η too big

Cheat sheet

Activation Functions — Derivatives

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \sigma'(z) = \sigma(x)(1 - \sigma(x)) = a(1 - a)$$

Tanh

$$tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

$$tanh'(z) = 1 - tanh^2(z) = 1 - a^2$$

ReLU

$$ReLU(z) = max(0, z)$$

$$ReLU'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

Leaky ReLU

$$f(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases}$$

$$f'(z) = \begin{cases} 1 & z > 0 \\ \alpha & z \leq 0 \end{cases}$$

Vector notation for NN training

Input:

$X_{(n,m)}$

$y_{(n,)}$

$n - number\ of\ data\ points$

$m - dimension\ of\ data\ point$

$l - hidden\ layer\ size$

Parameters:

$w_1(m,l)$

$b_1(1,l)$

$w_2(l,1)$

$b_2(1,1)$

Forward pass:

$$\underset{(n,m)}{X} \cdot \underset{(m,l)}{w_1} + \underset{(1,l)}{b_1} = \underset{(n,l)}{z_1}$$

$$\sigma(z_1) = \underset{(n,l)}{a_1}$$

$$\underset{(n,l)}{a_1} \cdot \underset{(l,1)}{w_2} + \underset{(1,1)}{b_2} = \underset{(n,1)}{z_2}$$

$$\sigma(z_2) = \underset{(n,1)}{a_1}$$

Backpropagation:

$$\frac{\partial L}{\partial z_2} = \underset{(n,1)(n,1)}{\hat{y} - y} = \underset{(n,1)}{dz_2}$$

$$\frac{\partial L}{\partial w_2} = \left(\underset{(l,n)}{a_1^T} \cdot \underset{(n,1)}{dz_2}\right)/n = \underset{(l,1)}{dw_2}$$

$$\frac{\partial L}{\partial b_2} = \left(\sum dz_2\right)/n = \underset{(1,1)}{db_2}$$

$$\frac{\partial L}{\partial a_1} = \underset{(n,1)}{dz_2} \cdot \underset{(1,l)}{w_2^T} = \underset{(n,l)}{da_1}$$

$$\frac{\partial L}{\partial z_1} = \underset{(n,l)\ (n,l)}{da_1 \circ \sigma'(a_1)} = \underset{(n,l)}{dz_1}$$

$$\frac{\partial L}{\partial w_1} = \left(\underset{(m,n)}{X^T} \cdot \underset{(n,l)}{dz_1}\right)/n = \underset{(m,l)}{dw_1}$$

$$\frac{\partial L}{\partial b_1} = \left(\sum dz_1\right)/n = \underset{(1,l)}{db_1}$$